

# An Implementation of the $n^{O(\sqrt{k})}$ Algorithm and of the EPTAS for 3-D Hypervolume Subset Selection

Technical report accompanying the `hssp3d` Python/Numba package

Michael Emmerich

University of Jyväskylä

September 22, 2026

## Abstract

The hypervolume subset selection problem (HSSP) asks for  $k$  out of  $n$  points that maximise the hypervolume indicator. In three dimensions the problem is NP-hard, and Bringmann, Cabello and Emmerich [1] gave the first exact algorithm that avoids the enumeration of all  $\binom{n}{k}$  subsets: a dynamic programme over planar separators of the projected optimal solution with running time  $n^{O(\sqrt{k})}$ . This report describes a complete implementation of that algorithm in Python with Numba-compiled kernels [4], states precisely where and why it deviates from the description in the paper, and validates it against brute-force enumeration on random samples of the DTLZ1 and DTLZ2 Pareto fronts with 15 points. The dynamic programme reproduced the brute-force optimum on every instance. As expected for an algorithm whose merit is asymptotic, it is orders of magnitude slower than enumeration for inputs of this size; the role of the constants hidden in the  $O(\sqrt{k})$  is discussed and measured, and an operation-count model locates the cross-over with enumeration. The efficient polynomial-time approximation scheme (EPTAS) of the same paper is implemented as well: its guarantees are confirmed against the optimum, and on inputs spanning many orders of magnitude it handles hundreds of points in seconds and returns a certified upper bound on the optimum.

## 1 Problem

For  $p \in \mathbb{R}_{>0}^3$  let  $\text{box}(p) = [0, p_1] \times [0, p_2] \times [0, p_3]$ , for a finite set  $Q$  let  $\mathcal{U}(Q) = \bigcup_{q \in Q} \text{box}(q)$  and  $\mu(Q) = \text{vol}(\mathcal{U}(Q))$ . Given a set  $P$  of  $n$  points and  $k \leq n$ , VOLUME SELECTION asks for

$$\text{VolSel}(P, k) = \max_{S \subseteq P, |S|=k} \mu(S)$$

together with a maximiser. For a minimisation problem with objective vectors  $f$  and reference point  $r$ , the substitution  $p = r - f$  turns the hypervolume indicator into  $\mu$ ; the

---

© 2026 Michael Emmerich. This report is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0), <https://creativecommons.org/licenses/by/4.0/>. The accompanying source code, <https://github.com/emmerichmtm/HSSP-3D>, is licensed under the MIT License.

package accepts both conventions. Dominated points never need to be selected and are removed beforehand, and ties in a coordinate are broken by the perturbation  $p + i\varepsilon(1, 1, 1)$  suggested in [1], so that  $P$  is an antichain in general position.

## 2 The algorithm of Bringmann, Cabello and Emmerich

We recall the construction of [1, Section 4] in the form in which it is implemented.

**Projection graph and triangulation.** The boundary of  $\mathcal{U}(Q)$ , projected onto the  $xy$ -plane, is a plane graph  $G(Q)$  with vertices  $v_q$  (the corner of  $q$ ),  $v_{q,q'}$  (a T-junction of two box boundaries),  $v_{x,q}, v_{y,q}$  (on the axes) and the origin. Every bounded face  $f(q, Q)$  is the part of the plane on which  $q$  is the highest box; it is bounded by a top segment, a right segment and a monotone staircase  $\gamma(q, Q)$ . The *canonical triangulation* adds diagonals from  $v_q$  to the interior vertices of  $\gamma(q, Q)$ , from the first interior vertex of  $\gamma$  to the vertices of the top segment and from the last one to those of the right segment. The outer face is triangulated towards the corners of a square  $\sigma \supset G(Q)$ . The result  $T(Q)$  is a triangulation of  $\sigma$  with  $O(|Q|)$  vertices. A polygonal domain  $D$  is  $Q$ -compliant if its boundary consists of edges of  $T(Q)$ , and  $P_\gamma$  denotes the points that define the vertices of a curve  $\gamma$ .

**Subproblems.** A valid tuple  $(S, D, \ell)$  consists of the points  $S$  selected so far, an  $S$ -compliant domain  $D$ , and a budget  $\ell$  of points that may still be chosen in  $P \cap (D \times \mathbb{R})$ :

$$\Phi_{\text{free}}(S, D, \ell) = \max\{\text{vol}(\mathcal{U}(S \cup Q) \cap (D \times \mathbb{R})) : Q \subseteq P \cap (D \times \mathbb{R}), |Q| \leq \ell\},$$

$$\Phi_{\text{comp}}(S, D, \ell) = \text{the same maximum over those } Q \text{ for which } D \text{ is } (S \cup Q)\text{-compliant.}$$

A *valid partition* of  $(S, D, \ell)$  is a family  $\{(S \cup S_0, D_i, \ell_i)\}_i$  with  $S_0 \subseteq P \cap D$ ,  $|S_0| = O(\sqrt{|S| + \ell})$ ,  $(S \cup S_0)$ -compliant domains  $D_i$  with disjoint interiors and union  $D$ ,  $\ell = |S_0| + \sum_i \ell_i$  and  $\ell_i \leq 2\ell/3$ . The algorithm evaluates

$$\Psi_{\text{comp}}(S, D, \ell) = \begin{cases} \Phi_{\text{comp}}(S, D, \ell) & \text{if } \ell \leq O(\sqrt{k}), \\ \max_{\pi \in \Pi(S, D, \ell)} \sum_{(S', D', \ell') \in \pi} \Psi_{\text{comp}}(S', D', \ell') & \text{otherwise.} \end{cases} \quad (1)$$

**Correctness and running time.** Every valid partition glues feasible solutions of its parts into a feasible solution of the whole (Lemma 4.3), so  $\Psi_{\text{comp}} \leq \Phi_{\text{free}}$ . Conversely, for the optimal  $Q^*$  of  $\Phi_{\text{comp}}(S, D, \ell)$ , Miller's theorem [2] yields a simple cycle  $\gamma$  in  $T(S \cup Q^*)$  with  $O(\sqrt{|S| + \ell})$  vertices that has at most  $2|Q^*|/3$  of the vertices  $v_q$ ,  $q \in Q^*$ , on either side;  $S_0 = P_\gamma \setminus S$  and the pieces into which  $\gamma$  cuts  $D$  form a valid partition whose value is at least  $\Phi_{\text{comp}}(S, D, \ell)$  (Lemma 4.4). Hence  $\Phi_{\text{comp}} \leq \Psi_{\text{comp}} \leq \Phi_{\text{free}}$  (Lemma 4.5), and since  $\Phi_{\text{comp}}(\emptyset, \sigma, k) = \Phi_{\text{free}}(\emptyset, \sigma, k) = \text{VolSel}(P, k)$  the top-level value is optimal. Along any chain of the recursion  $|S| = O(\sqrt{k})$ , which bounds the number of tuples and the work per tuple by  $n^{O(\sqrt{k})}$  (Theorem 4.6).

### 3 Implementation

The package `hssp3d` consists of Numba-compiled kernels (`_kernels.py`) for all geometric and combinatorial inner loops and a memoised Python recursion over the valid tuples (`solver.py`).

**A common vertex space.** Compliance compares edges of *different* triangulations  $T(Q)$  and  $T(Q')$ . Every vertex of every  $T(Q)$ ,  $Q \subseteq P$ , has the  $x$ -coordinate of some point (or 0, or a side of  $\sigma$ ) and likewise for  $y$ . Vertices are therefore identified by their pair of coordinate *ranks*, a global integer id on an  $(n+3) \times (n+3)$  grid. Edges and triangles of different triangulations become directly comparable, and the set  $P_v$  of points defining a vertex can be read off its id. Geometry (areas, orientation, point location) uses the real coordinates.

**The triangulation  $T(Q)$ .** `build_T` processes the points by decreasing  $z$ . For a point  $q$  the start of the visible part of its top edge is  $\max\{a_x : a_z > q_z, a_y > q_y\}$  (or the  $y$ -axis), analogously for the right edge; the staircase  $\gamma(q, Q)$  is traced through the two-dimensional skyline of the higher boxes, including the pieces that run along the axes; the T-junctions on the top (right) segment of  $q$  are the lower boxes whose right (top) edge ends there. The fans described in Section 2 are emitted as counter-clockwise triangles, each labelled with the point  $q$  owning the face. The cost is  $O(|Q|^2)$ . Inside a triangle of  $T(Q)$  the height of  $\mathcal{U}(Q)$  is constant, hence for every  $Q$ -compliant domain  $D$

$$\text{vol}(\mathcal{U}(Q) \cap (D \times \mathbb{R})) = \sum_{t \in T(Q), t \subseteq D} \text{area}(t) \cdot z_{\text{owner}(t)}.$$

Figure 1 shows an example. The unit tests verify for all subsets of several point sets that the output has  $2|V| - 6$  triangles (Euler’s formula for a triangulated square), that all triangles have positive area summing to the area of  $\sigma$ , that every interior edge is shared by exactly two triangles with opposite orientation, and that the formula above with  $D = \sigma$  equals the exact hypervolume computed by an independent sweep (which in turn is tested against inclusion–exclusion).

**Compliance test and restricted volume.** A domain is stored as the set of triangles of  $T(S)$  it consists of (a bit mask, which together with  $S$  and  $\ell$  forms the memoisation key) and handed to other triangulations as the list of its directed boundary edges, with  $D$  on the left. `eval_in_domain` enters the directed edges of  $T(S \cup Q)$  into a stamped table, checks that every boundary edge of  $D$  is present (this is exactly  $(S \cup Q)$ -compliance), and flood-fills the triangles of  $T(S \cup Q)$  inside  $D$  starting from the triangles to the left of the boundary edges. One evaluation costs  $O(|S \cup Q|^2)$ , dominated by `build_T`.

**Base case.** `base_case` is Lemma 4.2 verbatim: all  $Q \subseteq P \cap D$  with  $|Q| \leq \ell$  are enumerated, non-compliant ones are discarded, and the restricted volume is maximised.

**Valid partitions.** Following the remark in the proof of Theorem 4.6, partitions are generated from cycles rather than from arbitrary edge subsets. For every  $S_0 \subseteq P \cap D$  with  $|S_0| \leq \lceil c_s \sqrt{|S| + \ell} \rceil$  the triangulation  $T(S \cup S_0)$  is built; if  $D$  is not  $(S \cup S_0)$ -compliant,  $S_0$  is discarded (by Lemma 4.1 this cannot happen for the separator of the

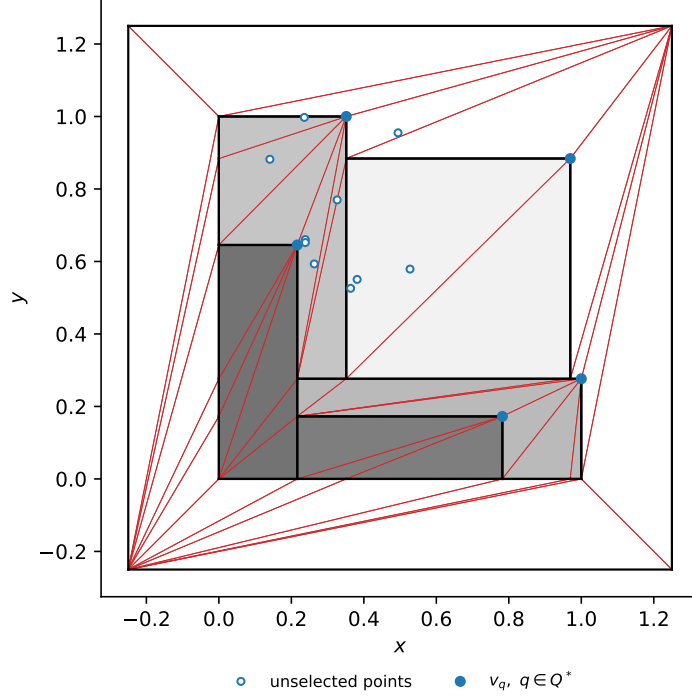


Figure 1: The triangulation  $T(Q^*)$  computed by `build_T` for the optimal 5-subset  $Q^*$  of a DTLZ2 instance with  $n = 15$  (seed 0,  $k = 5$  in Table 1), in normalised anchored-box coordinates  $p = r - f$ . Black: the projection graph  $G(Q^*)$ ; red: diagonals of the canonical triangulation and of the outer face; faces are shaded by the height  $z_q$  of their box (darker is higher). A separator is a short cycle in this graph.

optimal solution). Then all simple cycles  $\gamma$  of  $T(S \cup S_0)$  with  $|\gamma| \leq \lceil c_\gamma \sqrt{|S| + \ell} \rceil$  and  $P_\gamma \setminus S = S_0$  that cut  $D$  are enumerated by a depth-first search. Since the domains of all tuples are connected, a cycle cuts  $D$  if and only if it uses an edge interior to  $D$ ; each cycle is generated exactly once, from its lowest-ranked interior edge. The search is pruned by the breadth-first distance back to the start vertex and by the number of points of  $S_0$  that still have to be covered (a vertex is defined by at most two points). Distinct cycles that induce the same subdivision of  $D$  are merged. The connected components of the two sides are the domains  $D_i$ ; components without candidate points contribute the constant  $\text{vol}(\mathcal{U}(S \cup S_0) \cap (D_i \times \mathbb{R}))$ , and the budgets  $\ell_i \leq \lfloor 2\ell/3 \rfloor$  of the others are optimised by a max-plus knapsack over memoised values  $\Psi_{\text{comp}}(S \cup S_0, D_i, \ell_i)$ . A maximiser is assembled alongside the values, and the hypervolume of the returned subset is recomputed independently.

### Deviations from the paper.

1. *Constants.* The three constants of the  $O(\cdot)$ -notation (base-case threshold  $\lceil c_b \sqrt{k} \rceil$ , cycle length  $c_\gamma$ , separator size  $c_s$ ) are parameters; see Section 4.
2. *Trivial partitions.* The definition of a valid partition admits a single domain  $D_1 = D$  whenever  $|S_0| \geq \ell/3$ . These partitions can be switched off (`allow_trivial=False`, mode “sep” below), so that the separator machinery is tested in isolation.
3. *Budgets.* Budgets are clipped to the number of candidates of a domain and must

then add up exactly; this is without loss of generality since  $\Phi_{\text{comp}}$  is monotone in  $\ell$ .

4. *Branch and bound* (optional, **prune**).  $\Psi_{\text{comp}}(S, D, \ell)$  is initialised with a greedy compliant solution, and a set  $S_0$  (or a single partition) is skipped if  $\text{vol}(\mathcal{U}(S \cup S_0) \cap (D \times \mathbb{R}))$  plus the  $\ell - |S_0|$  largest marginal-gain bounds of the remaining candidates cannot beat the incumbent; this bound is valid by submodularity of  $\mu$ . Initialising with a feasible value keeps  $\Phi_{\text{comp}} \leq \Psi_{\text{comp}} \leq \Phi_{\text{free}}$  intact and pruning only removes partitions that cannot improve the value, so neither correctness nor the worst-case bound is affected.
5. *Fall-back*. If, with constants that are too small, no valid partition spends the whole budget, the returned set is completed greedily to  $k$  points.

## 4 Adherence to the $n^{O(\sqrt{k})}$ bound and the constants

**Observation 1.** *For every fixed choice of  $c_b, c_\gamma, c_s$  the implementation runs in time  $n^{O(\sqrt{k})}$ , and its output is a feasible  $k$ -subset whose hypervolume is a lower bound on  $\text{VolSel}(P, k)$ .*

The argument is that of Theorem 4.6. Budgets shrink by a factor  $2/3$  per level while  $|S|$  grows by at most  $c_s \sqrt{|S| + \ell}$ , so  $|S| = O(\sqrt{k})$  for all tuples. A tuple is determined by  $S$  ( $n^{O(\sqrt{k})}$  choices), a subset of the  $O(\sqrt{k})$  triangles of  $T(S)$  and  $\ell \leq k$ . A recursive tuple enumerates  $n^{O(\sqrt{k})}$  sets  $S_0$  and, in a graph with  $O(\sqrt{k})$  vertices, at most  $O(\sqrt{k})^{O(\sqrt{k})} \leq n^{O(\sqrt{k})}$  cycles of length  $O(\sqrt{k})$ ; a base case enumerates  $n^{O(\sqrt{k})}$  sets  $Q$ . All remaining work per item is polynomial. Feasibility of the output is Lemma 4.3 and needs no assumption on the constants.

*Optimality*, in contrast, is only guaranteed if the separator of Lemma 4.4 is among the enumerated cycles.  $T(Q)$  has  $3|Q| + 5 + c \leq 5|Q| + 5$  vertices, where  $c \leq 2|Q|$  counts the T-junctions, and its outer face is a quadrilateral, so Miller’s theorem gives  $|\gamma| \leq 4\sqrt{5(|S| + \ell)} + 5$  and  $|P_\gamma| \leq 2|\gamma|$ . These proven constants are available through the option **theory=True**. For  $|S| + \ell = 6$  they allow cycles with 23 vertices and  $|S_0| = \ell$ ; the bound  $2 \cdot 4\sqrt{5m} + 5$  on  $|S_0|$  drops below  $m = |S| + \ell$  only for  $m > 320$ . For every instance that can be solved in practice, the algorithm with proven constants therefore degenerates to (an expensive form of) enumeration. The experiments below use the small constants  $c_b = 1$ ,  $c_\gamma = 2.5$ ,  $c_s = 1$ , for which the recursion is genuinely exercised, and compare the result with brute force.

## 5 Experiments

**Setup.** Instances are  $n = 15$  points drawn uniformly at random from the Pareto fronts of three-objective DTLZ problems [3]: from the DTLZ2 front (the positive octant of the unit sphere, reference point  $(1.1, 1.1, 1.1)$ ) and from the DTLZ1 front (the simplex  $f_1 + f_2 + f_3 = 0.5$ , reference point  $(0.55, 0.55, 0.55)$ ). Random samples are in general position and mutually non-dominated. The reference solution enumerates all  $\binom{15}{k}$  subsets and evaluates each with an exact 3-D hypervolume sweep. Two modes of the dynamic programme are reported: “full” (all valid partitions, with pruning) and “sep” (only partitions induced by cycles that cut the domain, with pruning). A result counts as a match if the relative deviation from the brute-force optimum is below  $10^{-10}$ . All runs: Python 3.12, Numba 0.60, single thread of an AMD Ryzen 7 5800H.

Table 1: Hypervolume of the selected subsets,  $n = 15$ . BF: brute force; DP: separator dynamic programme in the two modes; the greedy  $(1 - 1/e)$ -approximation is listed for comparison.

| front | $k$ | seed | BF (optimum) | DP full    | DP sep     | greedy     | match |
|-------|-----|------|--------------|------------|------------|------------|-------|
| DTLZ1 | 3   | 0    | 0.09189681   | 0.09189681 | 0.09189681 | 0.08561125 | yes   |
| DTLZ1 | 3   | 1    | 0.09152458   | 0.09152458 | 0.09152458 | 0.08634246 | yes   |
| DTLZ1 | 3   | 2    | 0.08694869   | 0.08694869 | 0.08694869 | 0.08318900 | yes   |
| DTLZ1 | 4   | 0    | 0.10063965   | 0.10063965 | 0.10063965 | 0.09587732 | yes   |
| DTLZ1 | 4   | 1    | 0.09968444   | 0.09968444 | 0.09968444 | 0.09503020 | yes   |
| DTLZ1 | 4   | 2    | 0.09123758   | 0.09123758 | 0.09123758 | 0.08834534 | yes   |
| DTLZ1 | 5   | 0    | 0.10598140   | 0.10598140 | 0.10598140 | 0.10186775 | yes   |
| DTLZ1 | 5   | 1    | 0.10476263   | 0.10476263 | 0.10476263 | 0.10159050 | yes   |
| DTLZ1 | 5   | 2    | 0.09413690   | 0.09413690 | 0.09413690 | 0.09232524 | yes   |
| DTLZ2 | 3   | 0    | 0.33987549   | 0.33987549 | 0.33987549 | 0.33505362 | yes   |
| DTLZ2 | 3   | 1    | 0.35534692   | 0.35534692 | 0.35534692 | 0.35534692 | yes   |
| DTLZ2 | 3   | 2    | 0.32171969   | 0.32171969 | 0.32171969 | 0.32171969 | yes   |
| DTLZ2 | 4   | 0    | 0.38481586   | 0.38481586 | 0.38481586 | 0.38481586 | yes   |
| DTLZ2 | 4   | 1    | 0.40120109   | 0.40120109 | 0.40120109 | 0.40120109 | yes   |
| DTLZ2 | 4   | 2    | 0.36929633   | 0.36929633 | 0.36929633 | 0.35979760 | yes   |
| DTLZ2 | 5   | 0    | 0.42583056   | 0.42583056 | 0.42583056 | 0.42583056 | yes   |
| DTLZ2 | 5   | 1    | 0.43887675   | 0.43887675 | 0.43887675 | 0.43204158 | yes   |
| DTLZ2 | 5   | 2    | 0.40016993   | 0.40016993 | 0.40016993 | 0.39067120 | yes   |

**Correctness.** Table 1 lists the values for every instance and Table 2 the effort. In all 36 runs (18 instances with  $k \in \{3, 4, 5\}$ , two modes each) the dynamic programme returned a subset whose hypervolume equals the brute-force optimum; in 36 of the runs it is the identical subset. The plain greedy algorithm is sub-optimal on 13 of the 18 instances. Hence the agreement is not an artefact of the greedy incumbent used for pruning; moreover Table 3 repeats the comparison without pruning.

**Deep recursion.** With the default constants and  $k \leq 5$  the recursion has depth one: the separator splits the budget into parts of size at most  $\lfloor 2k/3 \rfloor \leq 3 = \lceil \sqrt{k} \rceil$ , which are base cases. To exercise nested separators, Table 3 lowers the base-case threshold to 1, disables trivial partitions and pruning, and uses  $n = 10$ ; every budget  $\ell \geq 2$  is then split by a separator, and domains of the second level are proper sub-polygons of  $\sigma$  that must remain compliant.

**Constants.** Table 4 varies  $c_s$  and  $c_\gamma$ . If only a single new point may lie on the separator ( $s_{\max} = 1$ ), no balanced compliant separator exists for these instances: the recursion places only  $k - 1$  points, the last point is added greedily, and the result is a lower bound that happens to be optimal on four of the six instances. From  $s_{\max} = 2$  on, all instances are solved optimally, already with cycles of length three; longer cycles multiply the number of partitions without changing the result. This illustrates the statement of Section 4: small constants preserve feasibility and the running-time bound, and whether they suffice for optimality is an empirical question below the proven values.

**Running time.** The algorithm is “galactic”: for  $n = 15$  brute force needs a few milliseconds, the dynamic programme seconds to minutes, and the gap widens with  $k$  in this range because the number of partitions of the root grows much faster than  $\binom{15}{k}$ . For

Table 2: Work of the dynamic programme, averaged over the seeds ( $n = 15$ ). “depth”: maximal recursion depth;  $|S|$ : largest set of pre-selected points in a tuple; “tuples”: memoised valid tuples  $(S, D, \ell)$ ; “partitions”: distinct valid partitions evaluated; “base sets”: subsets  $Q$  enumerated in base cases;  $\binom{n}{k}$ : subsets enumerated by brute force.

| front | $k$ | mode | match | depth | $ S $ | tuples  | partitions | base sets  | $\binom{n}{k}$ | $t_{\text{BF}}$ [ms] | $t_{\text{DP}}$ [s] |
|-------|-----|------|-------|-------|-------|---------|------------|------------|----------------|----------------------|---------------------|
| DTLZ1 | 3   | full | 3/3   | 1     | 1     | 2 176   | 881        | 81 982     | 455            | 0.2                  | 0.3                 |
| DTLZ1 | 3   | sep  | 3/3   | 1     | 1     | 2 161   | 881        | 80 427     | 455            | 0.2                  | 0.2                 |
| DTLZ1 | 4   | full | 3/3   | 1     | 2     | 25 012  | 9 911      | 831 758    | 1 365          | 0.8                  | 3.2                 |
| DTLZ1 | 4   | sep  | 3/3   | 1     | 2     | 24 917  | 9 911      | 823 048    | 1 365          | 0.7                  | 3.3                 |
| DTLZ1 | 5   | full | 3/3   | 1     | 3     | 372 295 | 129 256    | 15 471 001 | 3 003          | 1.7                  | 57.1                |
| DTLZ1 | 5   | sep  | 3/3   | 1     | 3     | 371 867 | 129 256    | 15 405 867 | 3 003          | 1.2                  | 57.7                |
| DTLZ2 | 3   | full | 3/3   | 1     | 1     | 1 865   | 701        | 62 289     | 455            | 0.2                  | 0.1                 |
| DTLZ2 | 3   | sep  | 3/3   | 1     | 1     | 1 854   | 701        | 61 053     | 455            | 0.1                  | 0.2                 |
| DTLZ2 | 4   | full | 3/3   | 1     | 2     | 13 192  | 5 158      | 395 291    | 1 365          | 0.6                  | 1.3                 |
| DTLZ2 | 4   | sep  | 3/3   | 1     | 2     | 13 147  | 5 158      | 391 121    | 1 365          | 0.5                  | 1.2                 |
| DTLZ2 | 5   | full | 3/3   | 1     | 3     | 171 326 | 53 553     | 9 244 585  | 3 003          | 1.1                  | 27.0                |
| DTLZ2 | 5   | sep  | 3/3   | 1     | 3     | 171 145 | 53 553     | 9 201 482  | 3 003          | 1.1                  | 26.6                |

Table 3: Forced deep recursion:  $n = 10$ , base-case threshold 1 ( $c_b = 0.5$ ), separators only, no pruning. Columns as in Table 2.

| front | $k$ | mode        | match | depth | $ S $ | tuples  | partitions | base sets | $\binom{n}{k}$ | $t_{\text{BF}}$ [ms] | $t_{\text{DP}}$ [s] |
|-------|-----|-------------|-------|-------|-------|---------|------------|-----------|----------------|----------------------|---------------------|
| DTLZ1 | 3   | sep/noprune | 3/3   | 2     | 2     | 67 936  | 120 531    | 445 723   | 120            | 0.1                  | 4.6                 |
| DTLZ1 | 4   | sep/noprune | 3/3   | 2     | 3     | 936 901 | 1 708 707  | 5 844 268 | 210            | 0.1                  | 65.1                |
| DTLZ2 | 3   | sep/noprune | 3/3   | 2     | 2     | 69 298  | 122 082    | 435 722   | 120            | 0.1                  | 4.6                 |
| DTLZ2 | 4   | sep/noprune | 3/3   | 2     | 3     | 935 333 | 1 712 349  | 5 720 839 | 210            | 0.2                  | 65.2                |

$n = 15$  and  $k = 6$ , the first case in which the default constants lead to two levels of separators, a run in mode “sep” was stopped unfinished after 90 minutes and is therefore not reported; nested separators are covered by Table 3 instead. The asymptotic advantage,  $n^{O(\sqrt{k})}$  versus  $n^{\Theta(k)}$ , requires the exponent  $c\sqrt{k}$  (with the constants of the separator theorem, of  $|P_\gamma| \leq 2|\gamma|$  and of the recursion) to fall below  $k$ , which happens far outside the range in which either method is feasible. The value of the implementation is therefore not speed: it makes the construction of [1] executable, confirms on concrete data that the compliance-based recursion is correct, and provides tested building blocks (the triangulation  $T(Q)$ , compliant domains, cycle-induced partitions) for future work on practical separator-based or approximate variants.

## 6 Is there an instance on which the algorithm wins?

**Not one that can be run.** In all measurements the dynamic programme is slower than enumeration by three to five orders of magnitude, and the ratio deteriorates with  $k$  (Table 2). We also tried the most favourable setting we could think of: “chain” instances ( $x$  increasing,  $y$  decreasing,  $z$  increasing along the points), whose projection graph consists of stacked strips with a balanced separator through two points at every level, solved with the smallest constants that work ( $|S_0| \leq 2$ ,  $|\gamma| \leq 6$ , no trivial partitions). For  $n = 16$ ,  $k = 6$  the run was stopped unfinished after ten minutes, while brute force enumerates the 8008 subsets in milliseconds. An honest example of an advantage can therefore only be

Table 4: Influence of the constants:  $n = 12$ ,  $k = 5$ , separators only, no pruning, six instances per row.  $s_{\max}$  and  $L$  are the resulting bounds on  $|S_0|$  and  $|\gamma|$  at the root; “gap”: largest relative deviation from the optimum; “padded”: largest number of points that had to be added greedily.

| $c_s$ | $c_\gamma$ | $s_{\max}$ | $L$ | match | gap [%] | padded | partitions | $t_{\text{DP}}$ [s] |
|-------|------------|------------|-----|-------|---------|--------|------------|---------------------|
| 0.4   | 1.0        | 1          | 3   | 4/6   | 0.54    | 1      | 121        | 0.1                 |
| 0.4   | 2.0        | 1          | 5   | 4/6   | 0.54    | 1      | 721        | 0.3                 |
| 0.4   | 3.0        | 1          | 7   | 4/6   | 0.54    | 1      | 1 681      | 0.6                 |
| 0.8   | 1.0        | 2          | 3   | 6/6   | 0.00    | 0      | 691        | 0.3                 |
| 0.8   | 2.0        | 2          | 5   | 6/6   | 0.00    | 0      | 7 026      | 3.1                 |
| 0.8   | 3.0        | 2          | 7   | 6/6   | 0.00    | 0      | 43 687     | 15.8                |
| 1.0   | 1.0        | 3          | 3   | 6/6   | 0.00    | 0      | 1 489      | 0.5                 |
| 1.0   | 2.5        | 3          | 6   | 6/6   | 0.00    | 0      | 86 652     | 20.4                |

Table 5: Smallest  $k \leq n/2$  for which the operation bound of the dynamic programme drops below the cost of brute force (“–”: never). “default”:  $c_b = 1$ ,  $c_\gamma = 2.5$ ,  $c_s = 1$ , every  $k$  examined; “proven”: the constants of Miller’s theorem, located by a geometric scan in steps of 4 % and bisection, hence to within about 4 %.  $c$  is the exponent constant implied at the crossing,  $\text{DP} = n^{c\sqrt{k}}$ .

| $n$       | default constants |                  |            |     | proven constants |                  |                 |       |  |
|-----------|-------------------|------------------|------------|-----|------------------|------------------|-----------------|-------|--|
|           | $k$               | $\text{DP} \leq$ | BF         | $c$ | $k$              | $\text{DP} \leq$ | BF              | $c$   |  |
| 15        | —                 | —                | —          | —   | —                | —                | —               | —     |  |
| $10^2$    | —                 | —                | —          | —   | —                | —                | —               | —     |  |
| $10^3$    | —                 | —                | —          | —   | —                | —                | —               | —     |  |
| $10^4$    | 278               | $10^{554}$       | $10^{555}$ | 8.3 | —                | —                | —               | —     |  |
| $10^6$    | 79                | $10^{360}$       | $10^{361}$ | 6.8 | —                | —                | —               | —     |  |
| $10^7$    | 70                | $10^{393}$       | $10^{394}$ | 6.7 | 115 097          | $10^{272\,868}$  | $10^{272\,870}$ | 114.9 |  |
| $10^9$    | 60                | $10^{456}$       | $10^{462}$ | 6.5 | 53 617           | $10^{252\,270}$  | $10^{252\,274}$ | 121.1 |  |
| $10^{12}$ | 49                | $10^{520}$       | $10^{529}$ | 6.2 | 34 931           | $10^{275\,650}$  | $10^{275\,650}$ | 122.9 |  |
| $10^{18}$ | 35                | $10^{591}$       | $10^{593}$ | 5.6 | 25 819           | $10^{362\,047}$  | $10^{362\,049}$ | 125.2 |  |
| $10^{30}$ | 29                | $10^{827}$       | $10^{842}$ | 5.1 | 20 651           | $10^{539\,390}$  | $10^{539\,397}$ | 125.1 |  |

given in terms of operation counts.

**Operation-count model.** The script `experiments/crossover_model.py` evaluates a worst-case upper bound on the number of elementary operations of the implementation and compares it with the exact cost  $\binom{n}{k}k^2$  of enumeration. Along the worst-case chain of the recursion ( $\ell_{i+1} = \lfloor 2\ell_i/3 \rfloor$ ,  $|S_{i+1}| = |S_i| + s_i$  with  $s_i$  the bound on  $|S_0|$  and  $L_i$  the bound on  $|\gamma|$  at level  $i$ ,  $V_i = 5(|S_i| + s_i) + 5$  vertices and  $F_i = 2V_i - 6$  triangles) it uses

$$\#\text{tuples}_{i+1} \leq \min \left\{ \#\text{tuples}_i \cdot \binom{n}{\leq s_i} V_i^{L_i} F_i k, \quad \binom{n}{|S_{i+1}|} 2^F k \right\},$$

$$\text{work per recursive tuple} \leq \binom{n}{\leq s_i} V_i^{L_i} F_i k^2, \quad \text{work per base tuple} \leq \binom{n}{\leq \ell_i} (|S_i| + \ell_i)^2,$$

i.e. the choices of  $S_0$ , of the cycle, of a component and of a budget. The bound is loose (for  $n = 15$ ,  $k = 5$  it gives  $10^{17}$  operations where of the order of  $10^{10}$  are executed), so the true cross-over may occur somewhat earlier than predicted; the brute-force count is exact.

Table 6: Operation counts for instances far beyond the cross-over (default constants).

| $n$    | $k$  | separator DP $\leq$ | brute force | fewer operations | factor      |
|--------|------|---------------------|-------------|------------------|-------------|
| $10^6$ | 50   | $10^{315}$          | $10^{239}$  | brute force      | $10^{76}$   |
| $10^6$ | 100  | $10^{443}$          | $10^{446}$  | DP               | $10^3$      |
| $10^6$ | 200  | $10^{656}$          | $10^{830}$  | DP               | $10^{174}$  |
| $10^6$ | 500  | $10^{1017}$         | $10^{1871}$ | DP               | $10^{854}$  |
| $10^6$ | 1000 | $10^{1433}$         | $10^{3438}$ | DP               | $10^{2005}$ |
| $10^9$ | 50   | $10^{438}$          | $10^{389}$  | brute force      | $10^{49}$   |
| $10^9$ | 100  | $10^{620}$          | $10^{746}$  | DP               | $10^{126}$  |
| $10^9$ | 200  | $10^{923}$          | $10^{1430}$ | DP               | $10^{507}$  |
| $10^9$ | 500  | $10^{1446}$         | $10^{3371}$ | DP               | $10^{1925}$ |
| $10^9$ | 1000 | $10^{2051}$         | $10^{6438}$ | DP               | $10^{4387}$ |

**Result.** Table 5 shows that for  $n \leq 1000$  the bound never drops below the cost of enumeration: since  $\binom{n}{k} \leq 2^n$ , brute force is simply not expensive enough when  $n$  is small, whatever  $k$ . The  $\sqrt{k}$  in the exponent pays off only when  $n \gg k \gg 1$ . An example where the algorithm has an advantage is  $n = 10^6$ ,  $k = 200$  (Table 6): at most  $10^{656}$  operations against  $10^{830}$ , a factor of  $10^{174}$ , growing to  $10^{2005}$  for  $k = 1000$ . The first such instances appear around  $n = 10^6$ ,  $k \approx 80$ , where both methods need about  $10^{360}$  operations; at the crossing the bound corresponds to  $n^{c\sqrt{k}}$  with  $c \approx 5$ –8. With the constants for which exactness is proven, the model shows no cross-over at all for  $n \leq 10^6$ , for any  $k \leq n/2$ . One first appears between  $n = 10^6$  and  $10^7$ , at  $k \approx 1.2 \cdot 10^5$ , and it moves down only slowly with  $n$ : to  $k \approx 5.4 \cdot 10^4$  at  $n = 10^9$  and  $k \approx 2.1 \cdot 10^4$  at  $n = 10^{30}$ , where both sides exceed  $10^{250\,000}$  operations. The implied constant rises to  $c \approx 115$ –125. Two caveats pull in opposite directions: the bound on the dynamic programme is loose, so the true cross-over of the implementation lies at or below the tabulated  $k$ , whereas proven exactness requires the larger separator, which moves it up. The advantage of the algorithm is thus real but purely asymptotic: it lies entirely in a regime in which neither method can be executed.

## 7 The approximation scheme

The package also contains the EPTAS of [1, Section 5] for  $d = 3$  (`solve_eptas` in `eptas.py`).

**Algorithm.** Let  $\beta = (1 - \varepsilon)^{-1/d}$ , let  $\tau$  be the smallest integer larger than  $d/\varepsilon$  and  $\lambda = \beta^g$  the smallest power of  $\beta$  larger than  $d/\varepsilon$ . The positive orthant is divided into *regions*  $\prod_i [\lambda^{x_i}, \lambda^{x_i+1})$ ; for an offset  $\bar{\ell} \in \{0, \dots, \tau - 1\}^d$  the regions with  $x_i \equiv \bar{\ell}_i \pmod{\tau}$  for some  $i$  form thick grid boundaries, and the remaining regions form grid cells of  $(\tau - 1)^d$  regions each. For every offset the scheme (2) deletes the points in the boundaries, (3) partitions the remaining points into cells, (4) rounds all coordinates down to powers of  $\beta$  and removes duplicates, (5) solves every cell exactly for all budgets  $k'$ , (6) distributes the budget,  $V(\bar{\ell}) = \max_{k' \leq k} \sum_i \text{VolSel}(\tilde{P}_i, k_i)$ , by dynamic programming, and (7) returns the best offset. The shifting argument (Lemma 5.2) loses a factor  $1 - \varepsilon$ , rounding (Lemma 5.3) another one, and treating cells as independent (Lemmas 5.4 and 5.5) a third one: with  $V = \max_{\bar{\ell}} V(\bar{\ell})$  and  $S$  the returned set of original points,

$$\mu(S) \geq (1 - \varepsilon) V, \quad V \geq (1 - \varepsilon)^2 \text{VolSel}(P, k), \quad \text{hence} \quad \mu(S) \geq (1 - \varepsilon)^3 \text{VolSel}(P, k). \quad (2)$$

Table 7: EPTAS against the brute-force optimum:  $n = 15$ ,  $k = 5$ , five instances per row. “guar.”: the guaranteed ratio  $(1 - \varepsilon)^3$ ; “min”/“mean”: observed  $\mu(S)/\text{OPT}$ ; “greedy”: smallest ratio of the greedy algorithm on the same instances; “UB/OPT”: smallest ratio of the certified upper bound to the optimum (must be  $\geq 1$ ); “cert.”: mean certified ratio  $\mu(S)/\text{UB}$ ; “part.”: distinct partitions among the  $\tau^3$  offsets; “cell”: largest number of rounded points in a cell.

| instances   | $\varepsilon$ | guar. | min    | mean   | greedy | UB/OPT | cert. | $\tau^3$ | part. | cell | $t$ [s] |
|-------------|---------------|-------|--------|--------|--------|--------|-------|----------|-------|------|---------|
| DTLZ1       | 0.5           | 0.125 | 0.9273 | 0.9635 | 0.9612 | 2.821  | 0.318 | 343      | 5     | 10   | 0.01    |
| DTLZ1       | 0.25          | 0.422 | 0.9797 | 0.9870 | 0.9612 | 1.565  | 0.626 | 2 197    | 3     | 14   | 0.09    |
| DTLZ1       | 0.1           | 0.729 | 0.9945 | 0.9980 | 0.9612 | 1.162  | 0.855 | 29 791   | 2     | 15   | 1.06    |
| DTLZ2       | 0.5           | 0.125 | 0.9401 | 0.9692 | 0.9763 | 2.870  | 0.322 | 343      | 17    | 14   | 0.02    |
| DTLZ2       | 0.25          | 0.422 | 0.9878 | 0.9939 | 0.9763 | 1.498  | 0.645 | 2 197    | 8     | 15   | 0.08    |
| DTLZ2       | 0.1           | 0.729 | 0.9970 | 0.9994 | 0.9763 | 1.163  | 0.855 | 29 791   | 8     | 15   | 1.09    |
| log-simplex | 0.5           | 0.125 | 0.9785 | 0.9950 | 1.0000 | 2.873  | 0.333 | 343      | 319   | 5    | 0.02    |
| log-simplex | 0.25          | 0.422 | 0.9963 | 0.9993 | 1.0000 | 1.447  | 0.653 | 2 197    | 1 489 | 10   | 0.15    |
| log-simplex | 0.1           | 0.729 | 0.9963 | 0.9993 | 1.0000 | 1.156  | 0.854 | 29 791   | 1 836 | 15   | 1.34    |

The second inequality makes  $V/(1 - \varepsilon)^2$  a *certified upper bound* on the unknown optimum. The implementation returns it together with  $S$ , so that the quality of  $S$  (or of any other solution, e.g. the greedy one) can be certified a posteriori; in three dimensions  $\mu(S)$  itself is computed exactly.

**Implementation.** Points are represented by their integer exponents  $e_i = \lfloor \log_\beta p_i \rfloor$  (computed with a correction step against rounding of the logarithm), so that region indices  $\lfloor e_i/g \rfloor$ , cell indices and rounded points are exact integer quantities. Step (5) enumerates subsets with a Numba kernel; two exact simplifications are applied: rounded points dominated by another rounded point of the same cell are dropped, and the solution of a cell is cached, because the same group of points forms a cell for many offsets, as are offsets that induce the same partition. Step (6) is the  $O(nk)$  dynamic programme of the paper (the knapsack-based improvement of Section 5.4 is not implemented). If fewer than  $k$  points are selected, the set is completed greedily, which can only increase  $\mu(S)$ . The exponential term  $2^{O(\varepsilon^{-2} \log 1/\varepsilon)^d}$  of the running time is the enumeration inside a cell: a cell spans  $(\tau - 1)g$  rounded values per coordinate, that is 48, 312 and 2910 for  $\varepsilon = 0.5, 0.25$  and  $0.1$ , or 5.6, 14 and 46 orders of magnitude. The implementation refuses (**CellTooLarge**) to enumerate more than  $5 \cdot 10^7$  subsets in one cell. The scheme is therefore practical either for small  $n$ , or for inputs whose coordinates span many orders of magnitude, so that cells are sparsely populated.

**Experiments.** Table 7 compares the scheme with the brute-force optimum on DTLZ instances as in Section 5 and on *log-simplex* instances: points  $10^u$  with  $u$  uniform on the plane  $u_1 + u_2 + u_3 = c$ ,  $|u_i| \leq 10$ ,  $c \in [-1, 1]$ , so that the coordinates span 20 orders of magnitude and the boxes have comparable volumes. The guarantee (2) and the validity of the upper bound hold on all 45 runs, as they must; the unit tests check them, including  $\mu(S) \geq (1 - \varepsilon)V$ , on further instances. The observed ratios ( $\geq 0.927$  for  $\varepsilon = 0.5$ ,  $\geq 0.994$  for  $\varepsilon = 0.1$ ) are far better than the worst-case guarantee, and for  $\varepsilon \leq 0.25$  the scheme is more reliable than greedy on the DTLZ instances. For the DTLZ fronts the surviving points share one cell (at most two for  $\varepsilon = 0.5$ ), so the scheme essentially amounts to deleting boundary points, rounding and enumeration.

Table 8: EPTAS on log-simplex instances far beyond enumeration (three instances per row).  $\binom{n}{k}$ : subsets brute force would have to enumerate; “cells”: largest number of non-empty cells over the offsets (mean over the instances); “cell”: largest number of rounded points in a cell; “vs. greedy”: mean  $\mu(S_{\text{EPTAS}})/\mu(S_{\text{greedy}})$ ; “cert.”: mean certified ratio  $\mu(S)/\text{UB}$ .

| $n$ | $k$ | $\binom{n}{k}$ | $\varepsilon$ | solved | cells | cell | vs. greedy | guar. | cert. | $t$ [s] | $t_{\text{greedy}}$ [ms] |
|-----|-----|----------------|---------------|--------|-------|------|------------|-------|-------|---------|--------------------------|
| 100 | 10  | $10^{13}$      | 0.5           | 3/3    | 67    | 3    | 0.9851     | 0.125 | 0.355 | 0.1     | 0                        |
| 100 | 10  | $10^{13}$      | 0.25          | 3/3    | 49    | 9    | 0.9982     | 0.422 | 0.644 | 0.6     | 0                        |
| 200 | 20  | $10^{27}$      | 0.5           | 3/3    | 114   | 6    | 0.9644     | 0.125 | 0.344 | 0.1     | 2                        |
| 200 | 20  | $10^{27}$      | 0.25          | 3/3    | 63    | 17   | 0.9982     | 0.422 | 0.650 | 2.2     | 2                        |
| 400 | 20  | $10^{33}$      | 0.5           | 3/3    | 184   | 7    | 0.9700     | 0.125 | 0.339 | 0.3     | 4                        |
| 400 | 20  | $10^{33}$      | 0.25          | 3/3    | 77    | 23   | 0.9956     | 0.422 | 0.645 | 95.8    | 4                        |

Table 8 uses log-simplex instances over 80 orders of magnitude with up to 400 points and  $k = 20$ . Here the input splits into roughly 50 to 190 cells with at most 23 rounded points, and the scheme needs between 0.1 seconds and a few minutes, whereas brute force would have to enumerate up to  $10^{33}$  subsets: this is a *measured* advantage over enumeration, in contrast to the exact algorithm (Section 6). Two caveats are in order. First, the greedy algorithm is faster still and its solutions are slightly better on these instances (by 0.2–3.6 % on average); widely spread boxes overlap little, which makes the instances easy. Second, the a-priori guarantee  $(1 - \varepsilon)^3$  is weak for the values of  $\varepsilon$  for which cells remain enumerable. What the scheme adds in practice is the certificate: for  $\varepsilon = 0.25$  it proves that the returned set, and hence also the better greedy set, is within a factor 0.64–0.65 of the optimum, slightly more than the  $1 - 1/e \approx 0.632$  known for greedy a priori, and for  $\varepsilon = 0.1$  it certifies about 0.85 on the instances of Table 7.

## 8 Conclusion

The  $n^{O(\sqrt{k})}$  algorithm for 3-D hypervolume subset selection has been implemented in full and agrees with brute-force enumeration on all tested instances of the DTLZ1 and DTLZ2 fronts, also when only cycle separators are used and when the recursion is forced to be deep. In terms of operation counts it overtakes enumeration only for instances such as  $n = 10^6$ ,  $k \geq 80$  (Section 6), far outside the range in which either method can be run. The EPTAS of [1, Section 5] has been implemented for  $d = 3$  as well; its guarantees were confirmed against the optimum, it handles wide-range instances with hundreds of points in seconds, and it returns a certified upper bound on the optimum. Source code, tests and the scripts that generate all tables of this report are contained in the repository.

## References

- [1] K. Bringmann, S. Cabello, M. T. M. Emmerich. Maximum volume subset selection for anchored boxes. In *33rd International Symposium on Computational Geometry (SoCG 2017)*, LIPIcs 77, 22:1–22:15, 2017. Full version: arXiv:1803.00849.
- [2] G. L. Miller. Finding small simple cycle separators for 2-connected planar graphs. *Journal of Computer and System Sciences*, 32(3):265–279, 1986.

- [3] K. Deb, L. Thiele, M. Laumanns, E. Zitzler. Scalable test problems for evolutionary multiobjective optimization. In *Evolutionary Multiobjective Optimization*, 105–145, Springer, 2005.
- [4] S. K. Lam, A. Pitrou, S. Seibert. Numba: a LLVM-based Python JIT compiler. In *Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC*, 2015.